

Лабораторная работа № 7. Введение в работу с данными

Абакумова Олеся Максимовна, НФИбд-02-22

Содержание

1	Цель работы	6
2	Задание	7
3	Выполнение лабораторной работы	8
3.1	Предварительные сведения	8
3.2	Julia для науки о данных	8
3.3	Считывание данных	8
3.4	Запись данных в файл	12
3.5	Словари	13
3.6	DataFrames	14
3.7	RDatasets	15
3.8	Работа с переменными отсутствующего типа (Missing Values)	17
3.9	FileIO	19
3.10	Обработка данных: стандартные алгоритмы машинного обучения в Julia	21
3.10.1	Кластеризация данных. Метод k-средних	21
3.10.2	Кластеризация данных. Метод k ближайших соседей	27
3.10.3	Обработка данных. Метод главных компонент	29
3.10.4	Обработка данных. Линейная регрессия	31
3.11	Задания для самостоятельного выполнения	34
4	Выводы	42

Список иллюстраций

3.1	Установка и подключение необходимых пакетов	9
3.2	Считывание данных и их запись в структуру	9
3.3	Функция определения по названию языка программирования года его создания	10
3.4	Функция определения по названию языка программирования года его создания (без учёта регистра)	11
3.5	Запись данных в формате CSV	12
3.6	Инициализация словаря	13
3.7	Подгрузка пакета и задача переменной со структурой DataFrame .	14
3.8	Получение доступа к столбцам по имени заголовка	15
3.9	Подгрузка пакета RDatasets и задача структуры	16
3.10	Определение типа переменной и вывод статистических данных .	17
3.11	Пример работы с данными с отсутствующим типом данных	18
3.12	Формирование таблицы данных и объединение их в один фрейм .	19
3.13	Подключение пакета FileIO и работа с изображением	20
3.14	Подключение пакетов и подгрузка houses.csv	21
3.15	Цены на недвижимость в зависимости от площади	22
3.16	Цены на недвижимость в зависимости от площади (исключены артефакты данных)	23
3.17	Подключение пакета Clustering и добавление в новый фрейм . . .	24
3.18	Подготовка данных и использование функции kmeans	25
3.19	Формирование нового фрейма	25
3.20	Пример кластеризации объектов недвижимости по географическому расположению	26
3.21	Пример кластеризации объектов недвижимости по почтовому индексу	27
3.22	Подключение и использование пакета NearestNeighbors	28
3.23	Определение соседей объекта недвижимости на графике и фильтрация по районам	29
3.24	Уменьшение размеров данных о цене и площади, и подключение пакета MultivariateStats	30
3.25	Определение главных компонент для данных по объектам недвижимости	31
3.26	Исходные данные	32
3.27	Линейная регрессия	33
3.28	Подобный код на языке Python	34
3.29	Задание № 1	35

3.30 Часть 1 задачи № 2	36
3.31 Часть 2 задачи № 2	37
3.32 Пункт а) задачи № 3	38
3.33 График пункта а) задачи № 3	39
3.34 Пункт b) задачи № 3	39
3.35 Пункт с) задачи № 3	40
3.36 Пункт d) задачи № 3	41

Список таблиц

1 Цель работы

Основной целью работы является освоение специализированных пакетов Julia для обработки данных.

2 Задание

1. Выполнить примеры, приведенные в лабораторной работе.
2. Выполнить задания для самостоятельного выполнения

3 Выполнение лабораторной работы

3.1 Предварительные сведения

Обработка и анализ данных, полученных в результате проведения исследований, — важная и неотъемлемая часть исследовательской деятельности. Большое значение имеет выявление определённых связей и закономерностей в имеющихся неструктурированных данных, особенно в данных больших размерностей. Выявленные в данных связи и закономерностей позволяет строить прогнозные модели с предполагаемым результатом. Для решения таких задач применяют методы из таких областей знаний как математическая статистика, программирование, искусственный интеллект, машинное обучение.

3.2 Julia для науки о данных

В Julia для обработки данных используются наработки из других языков программирования, в частности, из R и Python.

3.3 Считывание данных

Перед тем, как начать проводить какие-либо операции над данными, необходимо их откуда-то считать и возможно сохранить в определённой структуре. Довольно часто данные для обработки содержатся в csv-файле, имеющим текстовый формат, в котором данные в строке разделены, например, запятыми, и

соответствуют ячейкам таблицы, а строки данных соответствуют строкам таблицы. Также данные могут быть представлены в виде фреймов или множеств. В Julia для работы с такого рода структурами данных используют пакеты CSV, DataFrames, RDatasets, FileIO (рис. 3.1):

```
In [1]: # Обновление окружения:
using Pkg
Pkg.update

Out[1]: up (generic function with 7 methods)

In [2]: # Установка пакетов:
using Pkg
for p in ["CSV", "DataFrames", "RDatasets", "FileIO"]
    Pkg.add(p)
end
using CSV, DataFrames, DelimitedFiles
```

Рис. 3.1: Установка и подключение необходимых пакетов

Предположим, что у вас в рабочем каталоге с проектом есть файл с данными programminglanguages.csv, содержащий перечень языков программирования и год их создания. Тогда для заполнения массива данными для последующей обработки требуется считать данные из исходного файла и записать их в соответствующую структуру (рис. 3.2):

```
: # Считывание данных и их запись в структуру:
P = CSV.File("Downloads/programminglanguages.csv") |> DataFrame

: 73x2 DataFrame
   Row  year  language
   Int64  String31
1  1951  Regional Assembly Language
2  1952  Autocode
3  1954  IPL
4  1955  FLOW-MATIC
5  1957  FORTRAN
6  1957  COMTRAN
7  1958  LISP
8  1958  ALGOL 58
9  1959  FACT
10 1959  COBOL
11 1959  RPG
12 1962  APL
13 1962  Simula
   ⋮      ⋮
62 2003  Scala
63 2005  F#
64 2006  PowerShell
65 2007  Clojure
66 2009  Go
```

Рис. 3.2: Считывание данных и их запись в структуру

Далее приведём пример функции, в которой на входе указывается название языка программирования, а на выходе — год его создания (рис. 3.3):

```

julia> # Функция определения по названию языка программирования года его создания:
function language_created_year(P::DataFrame, language::String)
    loc = findfirst(P[:,2].==language)
    return P[loc,1]
end

julia> language_created_year (generic function with 1 method)

julia> # Пример вызова функции и определение даты создания языка Python:
language_created_year(P,"Python")

julia> 1991

julia> # Пример вызова функции и определение даты создания языка Julia:
language_created_year(P,"Julia")

julia> 2012

julia> language_created_year(P,"julia")
MethodError: no method matching getindex(::DataFrame, ::Nothing, ::Int64)
The function `getindex` exists, but no method is defined for this combination of argument types.

Closest candidates are:
  getindex(::DataFrame, ::typeof(!), ::Union{Signed, Unsigned})
    @ DataFrames C:\Users\user\.julia\packages\DataFrames\b4w9K\src\dataframe\dataframe.jl:548
  getindex(::DataFrame, ::Colon, ::Union{AbstractString, Signed, Symbol, Unsigned})
    @ DataFrames C:\Users\user\.julia\packages\DataFrames\b4w9K\src\dataframe\dataframe.jl:542
  getindex(::DataFrame, ::Integer, ::Union{Signed, Unsigned})
    @ DataFrames C:\Users\user\.julia\packages\DataFrames\b4w9K\src\dataframe\dataframe.jl:506
  ...

Stacktrace:
 [1] language_created_year(P::DataFrame, language::String)
    @ Main .\In[7]:4
 [2] top-level scope
    @ In[10]:1
 [3] eval(m::Module, e::Any)
    @ Core .\boot.jl:489

```

Рис. 3.3: Функция определения по названию языка программирования года его создания

В следующем примере (рис. 3.3) при вызове функции, в качестве аргумента которой указано слово `julia`, написанное со строчной буквы будет выдана ошибка, так как список не содержит таких данных.

Для того, чтобы убрать в функции зависимость данных от регистра, необходимо изменить исходную функцию следующим образом (рис. 3.4):

```

: # Функция определения по названию языка программирования
: # года его создания (без учёта регистра):
function language_created_year_v2(P,language::String)
    loc = findfirst(lowercase.(P[:,2]).==lowercase.(language))
    return P[loc,1]
end

: language_created_year_v2 (generic function with 1 method)

: # Пример вызова функции и определение даты создания языка julia:
language_created_year_v2(P,"julia")

: 2012

: # Построчное считывание данных с указанием разделителя:
Tx = readlm("Downloads/programminglanguages.csv", ',')

: 74x2 Matrix{Any}:
      "year"  "language"
1951      "Regional Assembly Language"
1952      "Autocode"
1954      "IPL"
1955      "FLOW-MATIC"
1957      "FORTRAN"
1957      "COMTRAN"
1958      "LISP"
1958      "ALGOL 58"
1959      "FACT"
1959      "COBOL"
1959      "RPG"
1962      "APL"
      ⋮
2003      "Scala"
2005      "F#"
2006      "PowerShell"
2007      "Clojure"
2009      "Go"
2010      "Rust"
2011      "Dart"
2011      "Kotlin"
2011      "Red"
2011      "Elixir"
2012      "Julia"
2014      "Swift"

```

Рис. 3.4: Функция определения по названию языка программирования года его создания (без учёта регистра)

Можно считывать данные построчно, с элементами, разделенными заданным разделителем (рис. 3.4).

3.4 Запись данных в файл

Предположим, что требуется записать имеющиеся данные в файл. Для записи данных в формате CSV можно воспользоваться следующим вызовом (рис. 3.5):

```
] : # Запись данных в CSV-файл:
CSV.write("Downloads/programming_languages_data2.csv", P)

] : "Downloads/programming_languages_data2.csv"

] : # Пример записи данных в текстовый файл с разделителем ',':
writedlm("Downloads/programming_languages_data.txt", Tx, ',')

] : # Пример записи данных в текстовый файл с разделителем '-':
writedlm("Downloads/programming_languages_data2.txt", Tx, '-')

] : # Построчное считывание данных с указанием разделителя:
P_new_delim = readldlm("Downloads/programming_languages_data2.txt", '-')

] : 74x2 Matrix{Any}:
      "year" "language"
1951      "Regional Assembly Language"
1952      "Autocode"
1954      "IPL"
1955      "FLOW-MATIC"
1957      "FORTRAN"
1957      "COMTRAN"
1958      "LISP"
1958      "ALGOL 58"
1959      "FACT"
1959      "COBOL"
1959      "RPG"
1962      "APL"
      ⋮
2003      "Scala"
2005      "F#"
2006      "PowerShell"
2007      "Clojure"
2009      "Go"
2010      "Rust"
2011      "Dart"
2011      "Kotlin"
2011      "Red"
2011      "Elixir"
2012      "Julia"
2014      "Swift"
```

Рис. 3.5: Запись данных в формате CSV

Можно проверить, используя `readldlm`, корректность считывания созданного текстового файла (рис. 3.5).

3.5 Словари

При работе с данными бывает удобно записать их в формате словаря. Предположим, что словарь должен содержать перечень всех языков программирования и года их создания, при этом при указании года выводить все языки программирования, созданные в этом году. При инициализации словаря можно задать конкретные типы данных для ключей и значений (рис. 3.6):

```
|: # Инициализация словаря:
dict = Dict{Integer,Vector{String}}{ }

|: Dict{Integer, Vector{String}}{ }

|: # Инициализация словаря:
dict2 = Dict{ }

|: Dict{Any, Any}{ }

|: # Заполнение словаря данными:
for i = 1:size(P,1)
    year,lang = P[i,:]
    if year in keys(dict)
        dict[year] = push!(dict[year],lang)
    else
        dict[year] = [lang]
    end
end

|: # Пример определения в словаре языков программирования, созданных в 2003 году:
dict[2003]

|: 2-element Vector{String}:
"Groovy"
"Scala"
```

Рис. 3.6: Инициализации словаря

Можно также инициировать пустой словарь, не задавая строго структуру (рис. 3.6).

В последнем случае словарь принимает ключи и значения любого типа. Далее требуется заполнить словарь ключами и годами, которые содержат все языки программирования, созданные в каждом году, в качестве значений, как показано на рис. 3.6.

В результате при вызове словаря можно, выбрав любой год, узнать, какие языки программирования были созданы в этом году.

3.6 DataFrames

Работа с данными, записанными в структуре DataFrame, позволяет использовать индексацию и получить доступ к столбцам по заданному имени заголовка или по индексу столбца. На примере с данными о языках программирования и годах их создания зададим структуру DataFrame (рис. 3.7):

```
|: # Подгружаем пакет DataFrames:
   using DataFrames

|: # Задаём переменную со структурой DataFrame:
   df = DataFrame(year = P[:,1], language = P[:,2])

|: 73×2 DataFrame
      Row  year  language
      Int64  String31
-----
1      1  1951  Regional Assembly Language
2      2  1952  Autocode
3      3  1954  IPL
4      4  1955  FLOW-MATIC
5      5  1957  FORTRAN
6      6  1957  COMTRAN
7      7  1958  LISP
8      8  1958  ALGOL 58
9      9  1959  FACT
10     10  1959  COBOL
11     11  1959  RPG
12     12  1962  APL
13     13  1962  Simula
      ⋮      ⋮      ⋮
62     62  2003  Scala
63     63  2005  F#
```

Рис. 3.7: Подгрузка пакета и задача переменной со структурой DataFrame

Если требуется получить доступ к столбцам по имени заголовка, то необходимо добавить к имени заголовка двоеточие (рис. 3.8):

```

julia> # Вывод всех значения столбца year:
df[:,year]

julia> 73-element Vector{Int64}:
 1951
 1952
 1954
 1955
 1957
 1957
 1958
 1958
 1959
 1959
 1962
 1962
  ⋮
 2003
 2005
 2006
 2007
 2009
 2010
 2011
 2011
 2011
 2011
 2012
 2014

julia> # Получение статистических сведений о фрейме:
describe(df)

julia> 2×7 DataFrame
 Row variable  mean  min    median  max    nmissing  eltype
 Symbol Union... Any    Union... Any    Int64      DataType
 1  year      1982.99 1951    1986.0   2014      0         Int64
 2  language           ALGOL 58           dBase III      0         String31

```

Рис. 3.8: Получение доступа к столбцам по имени заголовка

В Julia это означает, что имена заголовков обрабатываются как символы. Также следует иметь в виду, что вызов `df[1]` эквивалентен вызову `df[:,year]`. Пакет `DataFrames` предоставляет возможность с помощью `description` получить основные статистические сведения о каждом столбце во фрейме данных (рис. 3.8).

3.7 RDatasets

С данными можно работать также как с наборами данных через пакет `RDatasets` языка R (рис. 3.9):

```

: # Подгружаем пакет RDatasets:
  using RDatasets

: # Задаём структуру данных в виде набора данных:
  iris = dataset("datasets", "iris")

: 150×5 DataFrame
   Row  SepalLength  SepalWidth  PetalLength  PetalWidth  Species
   Float64      Float64      Float64      Float64      Cat...
1      5.1         3.5         1.4         0.2  setosa
2      4.9         3.0         1.4         0.2  setosa
3      4.7         3.2         1.3         0.2  setosa
4      4.6         3.1         1.5         0.2  setosa
5      5.0         3.6         1.4         0.2  setosa
6      5.4         3.9         1.7         0.4  setosa
7      4.6         3.4         1.4         0.3  setosa
8      5.0         3.4         1.5         0.2  setosa
9      4.4         2.9         1.4         0.2  setosa
10     4.9         3.1         1.5         0.1  setosa
11     5.4         3.7         1.5         0.2  setosa
12     4.8         3.4         1.6         0.2  setosa
13     4.8         3.0         1.4         0.1  setosa
⋮      ⋮          ⋮          ⋮          ⋮      ⋮
139    6.0         3.0         4.8         1.8  virginica

```

Рис. 3.9: Подгрузка пакета RDatasets и задача структуры

В данном случае набор данных содержит сведения о цветах. При этом следует иметь в виду, что данные, загруженные с помощью набора данных, хранятся в виде DataFrame. Пакет RDatasets также предоставляет возможность с помощью `description` получить основные статистические сведения о каждом столбце в наборе данных (рис. 3.10):

```

: # Определения типа переменной:
  typeof(iris)

: DataFrame

: describe(iris)

: 5×7 DataFrame
  Row variable   mean   min   median   max   nmissing   eltype
   Symbol Union... Any Union... Any Int64   DataType
1 SepalLength  5.84333  4.3   5.8     7.9     0   Float64
2 SepalWidth   3.05733  2.0   3.0     4.4     0   Float64
3 PetalLength  3.758    1.0   4.35    6.9     0   Float64
4 PetalWidth   1.19933  0.1   1.3     2.5     0   Float64
5 Species              setosa      virginica  0   CategoricalValue{String, UInt8}

```

Рис. 3.10: Определение типа переменной и вывод статистических данных

3.8 Работа с переменными отсутствующего типа (Missing Values)

Пакет DataFrames позволяет использовать так называемый «отсутствующий» тип. В операции сложения числа и переменной с отсутствующим типом значение также будет иметь отсутствующий тип. Приведём пример работы с данными, среди которых есть данные с отсутствующим типом (рис. 3.11):

```

|: # Отсутствующий тип:
|: a = missing
|: typeof(a)
|: Missing

|: # Пример операции с переменной отсутствующего типа:
|: a + 1
|: missing

|: # Определение перечня продуктов:
|: foods = ["apple", "cucumber", "tomato", "banana"]
|: # Определение калорий:
|: calories = [missing, 47, 22, 105]
|: 4-element Vector{Union{Missing, Int64}}:
|:      missing
|:      47
|:      22
|:      105

|: # Определение типа переменной:
|: typeof(calories)
|: Vector{Union{Missing, Int64}} (alias for Array{Union{Missing, Int64}, 1})

|: # Подключаем пакет Statistics:
|: using Statistics

|: # Определение среднего значения:
|: mean(calories)
|: missing

|: # Определение среднего значения без значений с отсутствующим типом:
|: mean(skipmissing(calories))
|: 58.0

```

Рис. 3.11: Пример работы с данными с отсутствующим типом данных

При попытке получить среднее значение калорий, ничего не получится из-за наличия переменной с отсутствующим типом. Для решения этой проблемы необходимо игнорировать отсутствующий тип (рис. 3.11).

Далее показано, как можно сформировать таблицы данных и объединить их в один фрейм (рис. 3.12):

```

: # Задание сведений о ценах:
prices = [0.85,1.6,0.8,0.6]
# Формирование данных о калориях:
dataframe_calories = DataFrame(item=foods,calories=calories)
# Формирование данных о ценах:
dataframe_prices = DataFrame(item=foods,price=prices)
# Объединение данных о калориях и ценах:
DF = innerjoin(dataframe_calories,dataframe_prices,on=:item)

: 4×3 DataFrame
  Row  item      calories  price
    String Int64? Float64
1  apple      missing    0.85
2  cucumber      47      1.6
3  tomato       22      0.8
4  banana      105      0.6

```

Рис. 3.12: Формирование таблицы данных и объединение их в один фрейм

3.9 FileIO

В Julia можно работать с так называемыми «сырыми» данными, используя пакет FileIO. Julia хранит изображение в виде множества цветов, можно определить тип и размер данных (рис. 3.13):

[illegible]

Рис. 3.13: Подключение пакета FileIO и работа с изображением

3.10 Обработка данных: стандартные алгоритмы машинного обучения в Julia

3.10.1 Кластеризация данных. Метод k-средних

Задача кластеризации данных заключается в формировании однородной группы упорядоченных по какому-то признаку данных. Метод k-средних позволяет минимизировать суммарное квадратичное отклонение точек кластеров от центров этих кластеров:

$$V = \sum_{i=1}^k \sum_{x \in S_i} (x - \mu_i)^2$$

где $S_i, i = 1, 2, \dots, k$ — полученные кластеры, k — число кластеров, μ_i — центры масс (главные точки или объекты кластера) всех векторов x из кластера S_i .

Рассмотрим задачу кластеризации данных на примере данных о недвижимости. Файл с данными `houses` S_i с SV содержит список транзакций с недвижимостью в районе Сакраменто, о которых было сообщено в течение определённого числа дней.

Сначала подключим необходимые для работы пакеты и подгрузим данные (рис. 3.14):

```

julia> using Plots

julia> # Загрузка данных:
houses = CSV.File("Downloads/houses.csv") |> DataFrame

julia> 985×12 DataFrame
          Row  street          city  zip  state  beds  baths  sq_ft  type  sale_date  price  latitude  longitude
          String  String15      Int64 String3 Int64 Int64 Int64 String15 String31      Int64 Float64  Float64
1      3526 HIGH ST  SACRAMENTO  95838  CA      2      1     836  Residential Wed May 21 00:00:00 EDT 59222  38.6319  -121.435
2      51 OMAHA CT  SACRAMENTO  95823  CA      3      1    1167  Residential Wed May 21 00:00:00 EDT 68212  38.4789  -121.431
3     2796 BRANCH ST  SACRAMENTO  95815  CA      2      1     796  Residential Wed May 21 00:00:00 EDT 68880  38.6183  -121.444
4     2805 JANETTE WAY  SACRAMENTO  95815  CA      2      1     852  Residential Wed May 21 00:00:00 EDT 69307  38.6168  -121.439
5     6001 MCMAHON DR  SACRAMENTO  95824  CA      2      1     797  Residential Wed May 21 00:00:00 EDT 81900  38.5195  -121.436
6     5828 PEPPERMILL CT  SACRAMENTO  95841  CA      3      1    1122  Condo      Wed May 21 00:00:00 EDT 89921  38.6626  -121.328
7     6048 OGDEN NASH WAY  SACRAMENTO  95842  CA      3      2    1104  Residential Wed May 21 00:00:00 EDT 90895  38.6817  -121.352

```

Рис. 3.14: Подключение пакетов и подгрузка `houses.csv`

Построим графики:

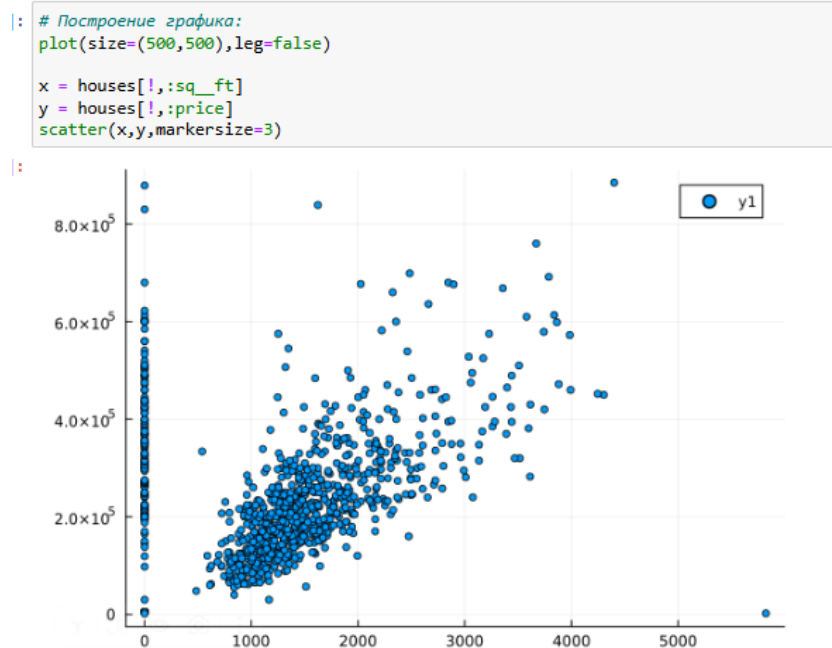


Рис. 3.15: Цены на недвижимость в зависимости от площади

Как видно из графика на рис. 3.15, имеются так называемые «артефакты», т.е. проявляются отсутствующие или невозможные сведения в исходных данных, например, цены на недвижимость нулевой площади.

Для того чтобы избавиться от такого эффекта, можно отфильтровать и исключить такие значения, получить более корректный график цен (рис. 3.16):

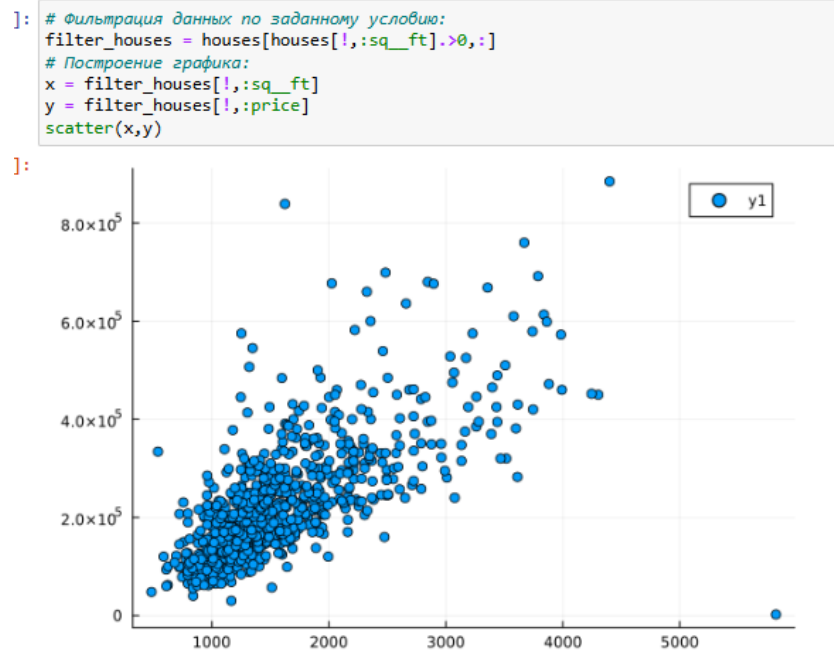


Рис. 3.16: Цены на недвижимость в зависимости от площади (исключены артефакты данных)

Используя для фильтрации значений функцию `by` пакета `DataFrames` и для вычисления среднего значения функцию `mean` пакета `Statistics`, можно посмотреть среднюю цену домов определённого типа (рис. 3.17):

```
julia> combine(groupby(filter_houses, :type), :price => mean)
julia> 3×2 DataFrame
  Row  type      price_mean
   String15  Float64
1  Residential  2.34802e5
2    Condo     1.34213e5
3 Multi-Family  2.24535e5
```

```
julia> # Подключение пакета Clustering:
Pkg.add("Clustering")
using Clustering

Resolving package versions...
Installed Clustering — v0.15.8
Installed NearestNeighbors — v0.4.24
Updating `C:\Users\user\.julia\environments\v1.12\Project.toml`
[aaaa29a8] + Clustering v0.15.8
Updating `C:\Users\user\.julia\environments\v1.12\Manifest.toml`
[aaaa29a8] + Clustering v0.15.8
[b8a86587] + NearestNeighbors v0.4.24
Precompiling packages...
4177.8 ms ✓ NearestNeighbors
2751.6 ms ✓ Clustering
2 dependencies successfully precompiled in 11 seconds. 633 already precompiled.
```

```
julia> # Добавление данных :latitude и :longitude в новый фрейм:
X = filter_houses[:, [:latitude, :longitude]]

julia> 814×2 DataFrame
  Row  latitude longitude
   Float64  Float64
1  38.6319   -121.435
2  38.4789   -121.431
3  38.6183   -121.444
4  38.6168   -121.439
```

Рис. 3.17: Подключение пакета Clustering и добавление в новый фрейм

Каждая функция хранится в виде строки X , но можно транспонировать получившуюся матрицу, чтобы иметь возможность работать с столбцами данных X . В качестве критерия для формирования кластеров данных и определения количества кластеров попробуем использовать количество почтовых индексов. Для определения k -среднего можно воспользоваться соответствующей функцией пакета Statistics (рис. 3.18):

```
X = Matrix(X)

814x2 Matrix:
38.6319 -121.435
38.4789 -121.431
38.6183 -121.444
38.6168 -121.439
38.5195 -121.436
38.6626 -121.328
38.6817 -121.352
38.5351 -121.481
38.6212 -121.271
38.7009 -121.443
38.6377 -121.452
38.4707 -121.459
38.6187 -121.436
:
38.7035 -121.375
38.7031 -121.235
38.3898 -121.446
38.8978 -121.325
38.4679 -121.445
38.4453 -121.442
38.4174 -121.484
38.4577 -121.36
38.4999 -121.459
38.7088 -121.257
38.417 -121.397
38.6552 -121.076

# Транспонирование матрицы с данными:
X = X'

2x814 adjoint(::Matrix{Float64}) with eltype Float64:
 38.6319  38.4789  38.6183  ...  38.7088  38.417  38.6552
-121.435 -121.431 -121.444 ... -121.257 -121.397 -121.076

# Задание количества кластеров:
k = length(unique(filter_houses[:, :zip]))

66

# Определение k-среднего:
C = kmeans(X, k)

KmeansResult{Matrix{Float64}, Float64, Int64}([38.39457549999994 38.04160333333333 ... 38.62510432258065 38.70436366666666
1.437903624999999 -121.197516 ... -121.43892561290322 -120.85083933333333], [65, 27, 65, 65, 39, 53, 3, 20, 8, 19 ... 1, 11
2, 38, 44, 45, 15, 61, 30], [6.273316466831602e-5, 0.00021722482051700354, 7.037215982563794e-5, 6.84302649460735e-5, 0.
4010815640, 0.0010271562594147017, 0.0003887753610070936, 0.0002632313297665663, 2.119247321167810e-5, 0.0001686658
```

Далее сформируем новый фрейм, включающий исходные данные о недвижимости и столбец с данными о назначенном каждому дому кластере (рис. 3.19):

```
df = DataFrame(cluster = C.assignments,city = filter_houses[:,city],
               latitude = filter_houses[:,latitude],longitude = filter_houses[:,longitude],zip = filter_houses[:,zip])
```

814x5 DataFrame

Row	cluster	city	latitude	longitude	zip
	Int64	String15	Float64	Float64	Int64
1	65	SACRAMENTO	38.6319	-121.435	95838
2	27	SACRAMENTO	38.4789	-121.431	95823
3	65	SACRAMENTO	38.6183	-121.444	95815
4	65	SACRAMENTO	38.6168	-121.439	95815
5	39	SACRAMENTO	38.5195	-121.436	95824
6	53	SACRAMENTO	38.6626	-121.328	95841
7	3	SACRAMENTO	38.6817	-121.352	95842
8	20	SACRAMENTO	38.5351	-121.481	95820
9	8	RANCHO CORDOVA	38.6212	-121.271	95670
10	19	RIO LINDA	38.7009	-121.443	95673
11	32	SACRAMENTO	38.6377	-121.452	95838
12	13	SACRAMENTO	38.4707	-121.459	95823
13	65	SACRAMENTO	38.6187	-121.436	95815
803	34	NORTH HIGHLANDS	38.7035	-121.375	95660
804	15	ORANGEVALE	38.7031	-121.235	95662
805	1	ELK GROVE	38.3898	-121.446	95757

Построим график, обозначив каждый кластер отдельным цветом (рис. 3.20):

```
clusters_figure = plot(legend = false)
for i = 1:k
    clustered_houses = df[df[:,cluster].== i,:]
    xvals = clustered_houses[:,latitude]
    yvals = clustered_houses[:,longitude]
    scatter!(clusters_figure,xvals,yvals,markersize=4)
end
xlabel!("Latitude")
ylabel!("Longitude")
title!("Houses color-coded by cluster")
display(clusters_figure)
```

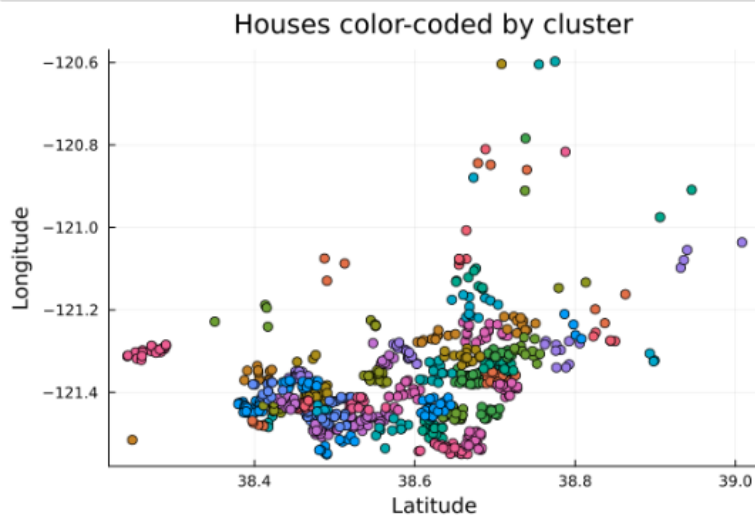


Рис. 3.20: Пример кластеризации объектов недвижимости по географическому расположению

Построим график, раскрасив кластеры по почтовому индексу (рис. 3.21):



Рис. 3.21: Пример кластеризации объектов недвижимости по почтовому индексу

3.10.2 Кластеризация данных. Метод k ближайших соседей

Данный метод заключается в отнесении объекта к тому из известных классов, который является наиболее распространённым среди k соседей данного элемента. В случае использования метода для регрессии, объекту присваивается среднее значение по k ближайшим к нему объектам. Рассмотрим использование метода k ближайших соседей на примере того же файла с данными об объектах недвижимости в Сакраменто. Подключим необходимый пакет, найдем k -среднее и определим ближайших соседей (рис. 3.22):

```

Pkg.add("NearestNeighbors")
using NearestNeighbors

Resolving package versions...
Updating `C:\Users\user\.julia\environments\v1.12\Project.toml`
[b8a86587] + NearestNeighbors v0.4.24
Manifest No packages added to or removed from `C:\Users\user\.julia\environments\v1.12\Ma

knearest = 10
id = 70
point = X[:,id]

2-element Vector{Float64}:
 38.44004
-121.421012

# Поиск ближайших соседей:
kdtree = KDTree(X)
idxs, dists = knn(kdtree, point, knearest, true)

([70, 764, 196, 125, 557, 368, 415, 92, 112, 683], [0.0, 0.006264891539364138, 0.008253202590
164073548370186, 0.009405065124697706, 0.00992175972295076, 0.009941028618812013, 0.010332637

```

Рис. 3.22: Подключение и использование пакета NearestNeighbors

Отообразим на графике соседей выбранного объекта недвижимости и отфильтруем по районам соседних домов (рис. 3.23):

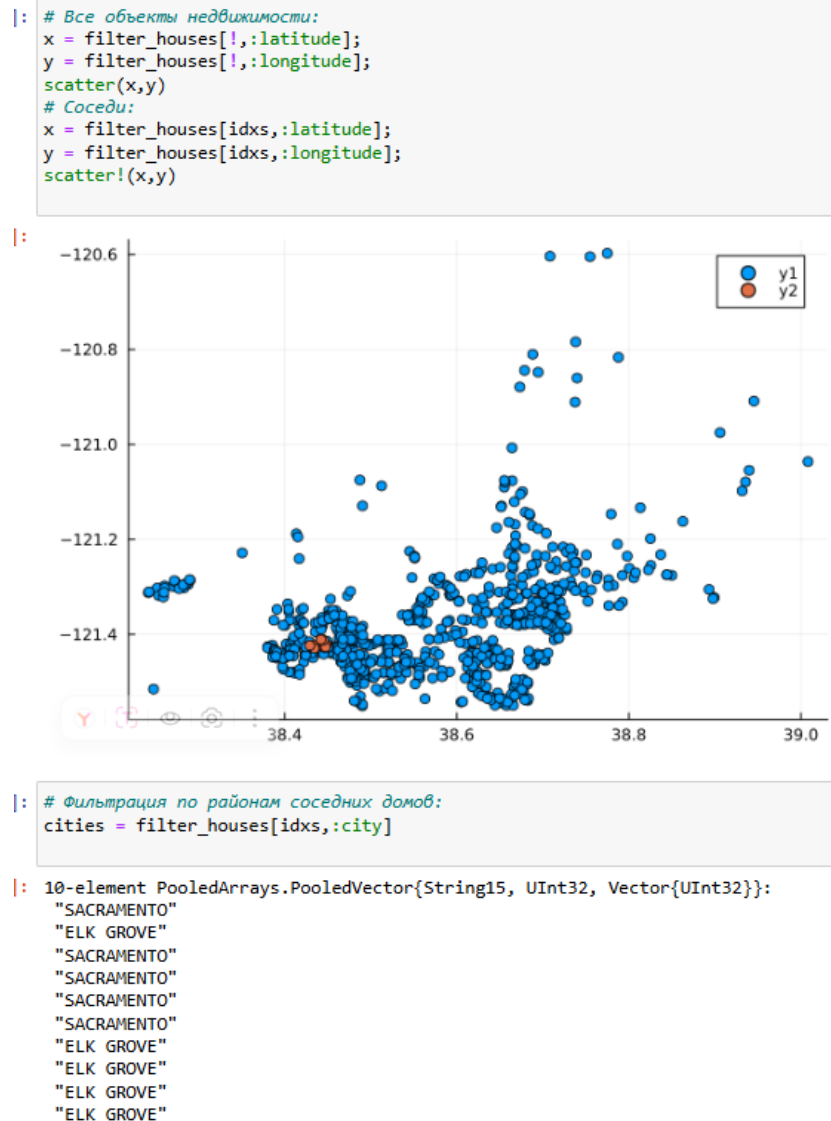


Рис. 3.23: Определение соседей объекта недвижимости на графике и фильтрация по районам

3.10.3 Обработка данных. Метод главных компонент

Метод главных компонент (Principal Components Analysis, PCA) позволяет уменьшить размерность данных, потеряв наименьшее количество полезной информации. Метод имеет широкое применение в различных областях знаний, например, при визуализации данных, компрессии изображений, в эконометри-

ке, некоторых гуманитарных предмет- ных областях, например, в социологии или в политологии.

На примере с данными о недвижимости попробуем уменьшить размеры данных о цене и площади из набора данных домов. Далее подключим пакет MultivariateStats, чтобы использовать метод главных компонент (рис. 3.24):

```
: # Выбор столбцов
F = filter_houses[:, [:sq_ft, :price]]

# Преобразование в матрицу и транспонирование
F = Matrix(F)'

: 2x814 adjoint(::Matrix{Int64}) with eltype Int64:
      836   1167    796    852    797   1122   ...   1477    1216    1685    1362
59222  68212  68880  69307  81900  89921      234000  235000  235301  235738

: # Подключение пакета MultivariateStats:
import Pkg
Pkg.add("MultivariateStats")
using MultivariateStats

Resolving package versions...
Installed MultivariateStats - v0.10.3
Installed Arpack - v0.5.4
Installed Arpack_jll - v3.5.1+1
Installing 1 artifacts
Installed artifact Arpack 239.1 KiB
Updating `C:\Users\user\.julia\environments\v1.12\Project.toml`
[6f286f6a] + MultivariateStats v0.10.3
Updating `C:\Users\user\.julia\environments\v1.12\Manifest.toml`
[7d9fca2a] + Arpack v0.5.4
[6f286f6a] + MultivariateStats v0.10.3
⚠ [68821587] + Arpack_jll v3.5.1+1
Info Packages marked with ⚠ have new versions available but compatibility const
see why use `status --outdated -m`
Precompiling packages...
1824.3 ms ✓ Arpack_jll
1182.3 ms ✓ Arpack
2846.0 ms ✓ MultivariateStats
3 dependencies successfully precompiled in 11 seconds. 635 already precompiled.

: # Приведение типов данных к распределению для PCA:
M = fit(PCA, F)

: PCA(indim = 2, outdim = 1, principalratio = 0.9999840784692097)

Pattern matrix (unstandardized loadings):
_____
PC1
_____
1  460.52
2   1.19826e5
```

Рис. 3.24: Уменьшение размеров данных о цене и площади, и подключение пакета MultivariateStats

Использована функция `fit` и приведём имеющийся набор данных к распределению, к которому можно применить метод главных компонент (PCA).

Далее воспользуемся функцией `reconstruct`, чтобы выделить данные с главными компонентами в отдельную переменную `X1`, значения которой в послед-

ствии можно вывести на графике (рис. 3.25):

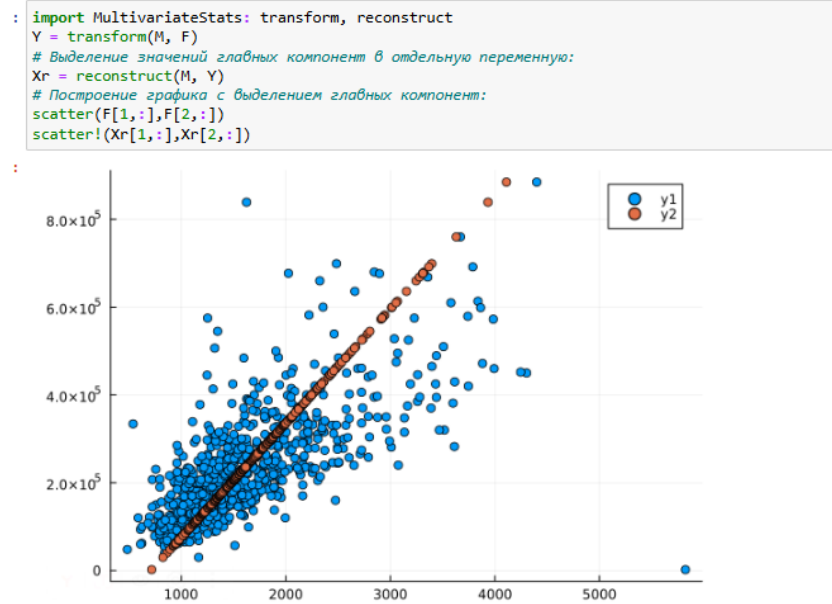


Рис. 3.25: Определение главных компонент для данных по объектам недвижимости

3.10.4 Обработка данных. Линейная регрессия

Регрессионный анализ представляет собой набор статистических методов исследования влияния одной или нескольких независимых переменных (регрессоров) на зависимую (критериальная) переменную. Терминология зависимых и независимых переменных отражает лишь математическую зависимость переменных, а не причинно-следственные отношения.

Наиболее распространённый вид регрессионного анализа — линейная регрессия, когда находят линейную функцию, которая согласно определённым математическим критериям наиболее соответствует данным.

Зададим случайный набор данных (можно использовать и полученные экспериментальным путём какие-то данные). Попробуем найти для данных лучшее соответствие (рис. 3.26):

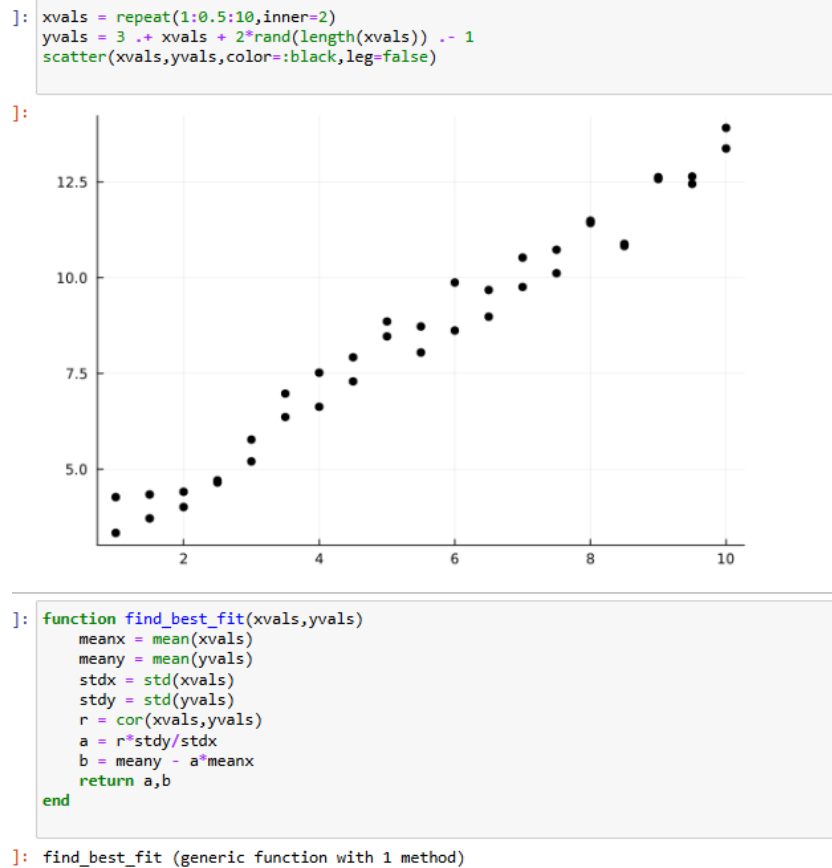


Рис. 3.26: Исходные данные

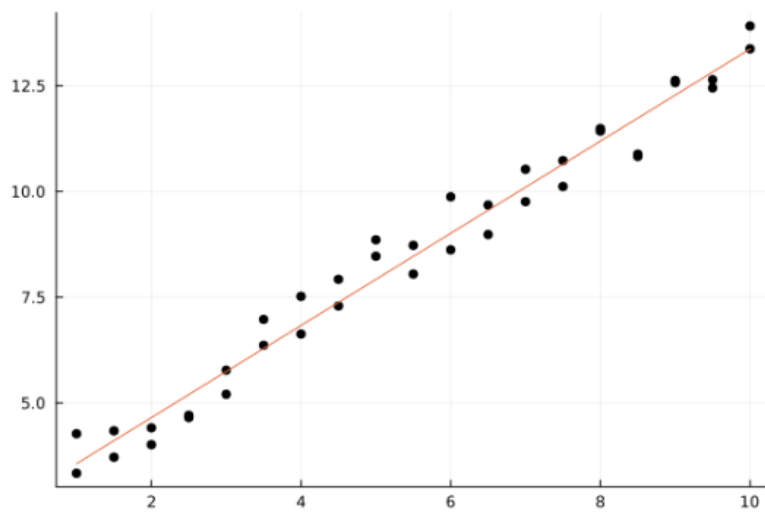
Выше также определена функция линейной регрессии (рис. 3.26).

Применим функцию линейной регрессии для построения соответствующего графика значений, сгенерируем больший набор данных (рис. 3.27):

```

a,b = find_best_fit(xvals,yvals)
ynew = a * xvals .+ b
plot!(xvals,ynew)

```



```

xvals = 1:100000;
xvals = repeat(xvals,inner=3);
yvals = 3 .+ xvals + 2*rand(length(xvals)) .- 1;

```

```
@show size(xvals)
```

```

size(xvals) = (300000,)
(300000,)

```

```
@show size(yvals)
```

```

size(yvals) = (300000,)
(300000,)

```

Рис. 3.27: Линейная регрессия

Определим, сколько времени потребуется, чтобы найти соответствие этим данным, для сравнения реализуем подобный код на языке Python (рис. 3.28):

```

|: @time a,b = find_best_fit(xvals,yvals)

0.055542 seconds (33.27 k allocations: 1.660 MiB, 95.94% compilation time)

|: (0.9999999674669128, 3.0012737458528136)

|: Pkg.add("PyCall")
Pkg.add("Conda")
using PyCall
using Conda

Resolving package versions...
Project No packages added to or removed from `C:\Users\user\.julia\environments\v1.
Manifest No packages added to or removed from `C:\Users\user\.julia\environments\v1.
Resolving package versions...
Project No packages added to or removed from `C:\Users\user\.julia\environments\v1.
Manifest No packages added to or removed from `C:\Users\user\.julia\environments\v1.

|: py"""
import numpy as np

def find_best_fit_python(xvals, yvals):
    meanx = np.mean(xvals)
    meany = np.mean(yvals)
    stdx = np.std(xvals)
    stdy = np.std(yvals)
    r = np.corrcoef(xvals, yvals)[0][1]
    a = r * stdy / stdx
    b = meany - a * meanx
    return float(a), float(b)
"""

find_best_fit_python = py"find_best_fit_python"
xpy = PyObject(xvals)
ypy = PyObject(yvals)
@time a,b = find_best_fit_python(xpy,ypy)

0.143657 seconds (138.93 k allocations: 6.850 MiB, 85.13% compilation time)

|: (0.9999999674669119, 3.0012737458964693)

|: Pkg.add("BenchmarkTools")
using BenchmarkTools
@btime a,b = find_best_fit_python(xvals,yvals)
@btime a,b = find_best_fit(xvals,yvals)

Resolving package versions...

```

Рис. 3.28: Подобный код на языке Python

3.11 Задания для самостоятельного выполнения

1. Загрузите

`using RDatasets`

`iris = dataset("datasets", "iris")`

Используйте `Clustering.jl` для кластеризации на основе k-средних. Сделайте точечную диаграмму полученных кластеров. Подсказка: вам нужно будет проиндексировать фрейм данных, преобразовать его в массив и транспонировать.

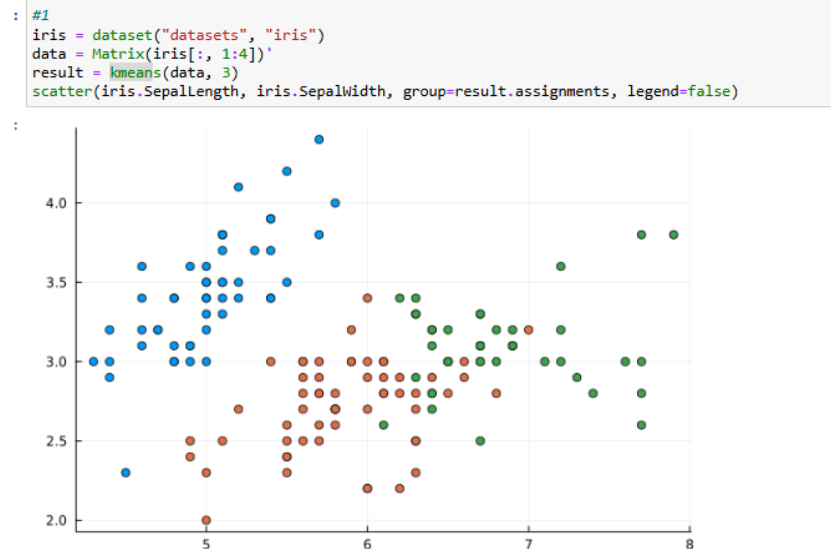


Рис. 3.29: Задание № 1

2.

Часть 1

```
X = randn(1000, 3)
```

```
a0 = rand(3)
```

```
y = X * a0 + 0.1 * randn(1000);
```

Часть 2

```
X = rand(100);
```

```
y = 2X + 0.1 * randn(100);
```

Часть 1 Пусть регрессионная зависимость является линейной. Матрица наблюдений факторов X имеет размерность $N \times 3$ `randn(N, 3)`, массив результатов $N \times 1$, регрессионная зависимость является линейной. Найдите МНК-оценку для линейной модели.

- Сравните свои результаты с результатами использования `llsq` из `MultivariateStats.jl` (посмотрите документацию).
- Сравните свои результаты с результатами использования регулярной регрессии наименьших квадратов из `GLN.jl`.

Подсказка. Создайте матрицу данных X_2 , которая добавляет столбец единиц в начало матрицы данных, и решите систему линейных уравнений. Объясните с помощью теоретических выкладок.

Часть 2 Найдите линию регрессии, используя данные (X, y) . Постройте график (X, y) , используя точечный график. Добавьте линию регрессии, используя `abline`!. Добавьте заголовок «График регрессии» и подпишите оси x и y .

```
#2
## часть 1
N = 1000
X = randn(N, 3)
a0 = rand(3)
y = X * a0 + 0.1 * randn(N)

XZ = hcat(ones(N), X)
a_mnk = (XZ' * XZ) \ (XZ' * y)
println("МНК-оценка (вручную): ", a_mnk)

a_llsq = llsq(X, y, bias=true)
println("МНК-оценка (MultivariateStats.jl, llsq): ", a_llsq)

df = DataFrame(X1=X[:,1], X2=X[:,2], X3=X[:,3], y=y)
model_glm = lm(@formula(y ~ X1 + X2 + X3), df)
a_glm = coef(model_glm)
println("МНК-оценка (GLM.jl): ", a_glm)

println("\nСравнение результатов:")
println("Разница между ручным МНК и llsq: ", norm(a_mnk - a_llsq))
println("Разница между ручным МНК и GLM: ", norm(a_mnk - a_glm))

МНК-оценка (вручную): [1.7187469083014937e-5, 0.3443082335329195, 0.27235378900307855, 0.3056438934715859]
МНК-оценка (MultivariateStats.jl, llsq): [0.34430823353291945, 0.27235378900307855, 0.305643893471586, 1.7187469083014192e-5]
МНК-оценка (GLM.jl): [1.7187469083015368e-5, 0.34430823353291945, 0.2723537890030785, 0.3056438934715859]

Сравнение результатов:
Разница между ручным МНК и llsq: 0.46715059773566936
Разница между ручным МНК и GLM: 7.850580216696627e-17
```

Рис. 3.30: Часть 1 задачи № 2

```

|: # Часть 2
X = rand(100)
y = 2 .* X .+ 0.1 * randn(100)

X2 = hcat(ones(100), X)
a_mnk_id = (X2' * X2) \ (X2' * y)
println("Оценки коэффициентов (одномерная регрессия): ", a_mnk_id)

p = scatter(X, y, label="Данные", title="График регрессии", xlabel="x", ylabel="y")
x_line = range(minimum(X), maximum(X), length=100)
y_line = a_mnk_id[1] .+ a_mnk_id[2] .* x_line
plot!(p, x_line, y_line, label="Линия регрессии", color=:red, linewidth=2)
display(p)

```

Оценки коэффициентов (одномерная регрессия): [0.004404848898564495, 1.9851621150224572]

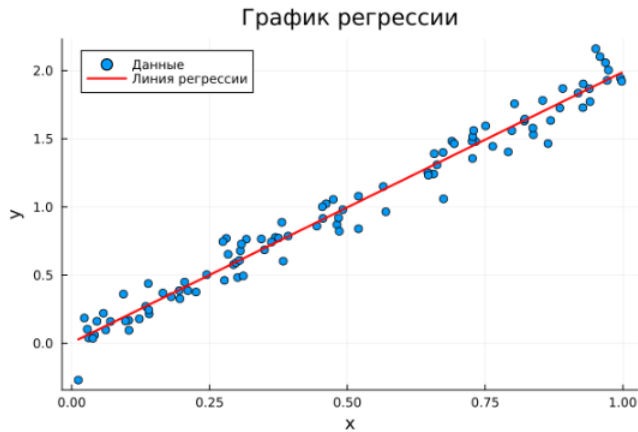


Рис. 3.31: Часть 2 задачи № 2

3. Постройте траекторию возможных цен на акции:

- S — начальная цена акции;
- T — длина биномиального дерева в годах;
- n — количество периодов;
- $h = \frac{T}{n}$ — длина одного периода;
- σ — волатильность акции;
- r — годовая процентная ставка;
- $u = \exp(rh + \sigma\sqrt{h})$;
- $d = \exp(rh - \sigma\sqrt{h})$;
- $p^* = \frac{\exp(rh) - d}{u - d}$.

a) Пусть $S = 100$, $T = 1$, $n = 10000$, $\sigma = 0.3$ и $r = 0.08$. Попробуйте построить траекторию курса акций. Функция `rand()` генерирует случайное число от 0 до 1. Вы можете использовать функцию построения графика из библиотеки графиков.

b) Создайте функцию `createPath(S::Float64, r::Float64, sigma::Float64, T::Float64, n::Int64)`, которая создает траекторию цены акции с учетом начальных параметров. Используйте `createPath`, чтобы создать 10 разных траекторий и построить их все на одном графике.

c) Распараллельте генерацию траектории. Можете использовать `Threads.@threads`, `map` и `@parallel`.

d) Пусть $S = 100$, $T = 1$, $n = 10000$, $\sigma = 0.3$ и $r = 0.08$. Попробуйте построить траекторию курса акций. Функция `rand()` генерирует случайное число от 0 до 1. Вы можете использовать функцию построения графика из библиотеки графиков.

```
#3
#a)
using Random
function createPath(S::Float64, r::Float64, sigma::Float64, T::Float64, n::Int64)
    h = T / n
    u = exp(r * h + sigma * sqrt(h))
    d = exp(r * h - sigma * sqrt(h))
    p_star = (exp(r * h) - d) / (u - d)

    path = Vector{Float64}(undef, n + 1)
    path[1] = S

    for i in 2:(n + 1)
        if rand() < p_star
            path[i] = path[i - 1] * u
        else
            path[i] = path[i - 1] * d
        end
    end

    return path
end

S = 100.0
T = 1.0
n = 10000
sigma = 0.3
r = 0.08

path_single = createPath(S, r, sigma, T, n)
time_axis = range(0, T, length = n + 1)

plot(time_axis, path_single, xlabel = "Time (years)", ylabel = "Stock Price",
     title = "Одна траектория", legend = false)
```

Рис. 3.32: Пункт а) задачи № 3

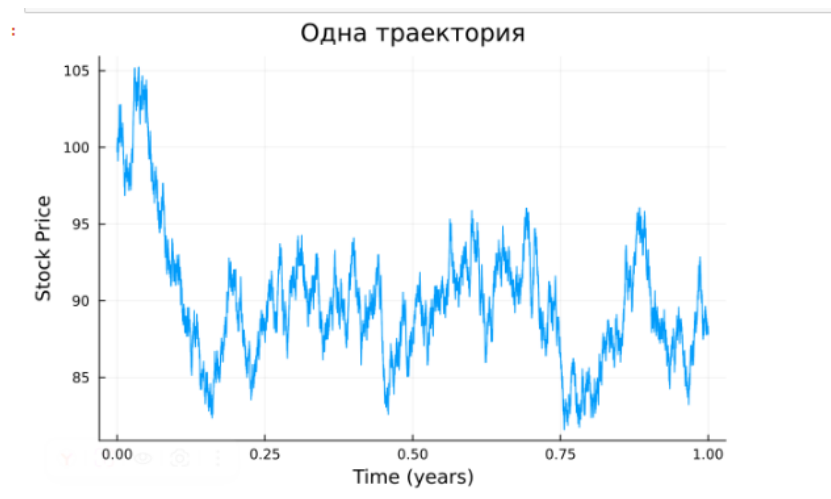


Рис. 3.33: График пункта а) задачи № 3

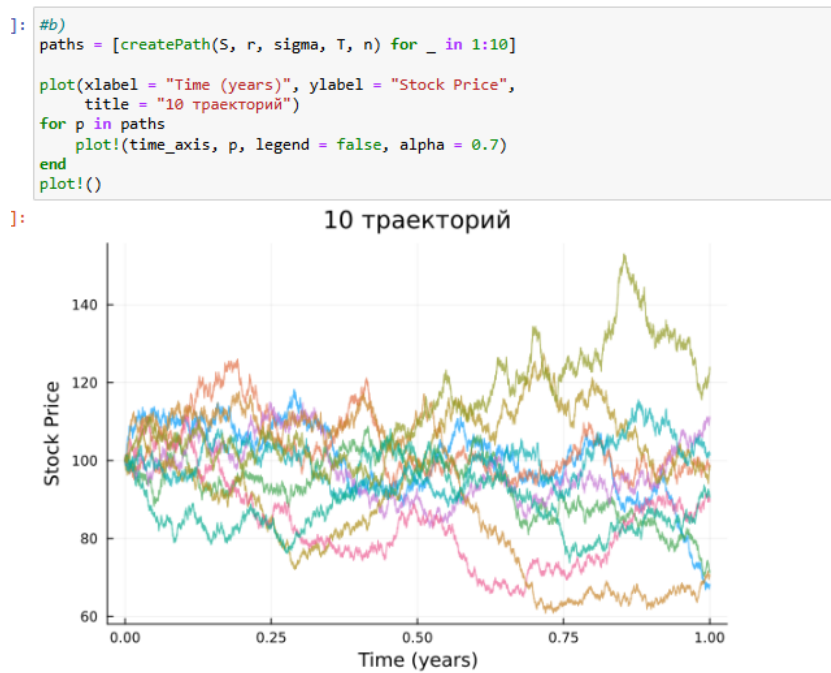


Рис. 3.34: Пункт b) задачи № 3

```

using Distributed, SharedArrays
#c)

function createPathParallel(S::Float64, r::Float64, sigma::Float64, T::Float64, n::Int64, numPaths)
    h = T / n
    u = exp(r * h + sigma * sqrt(h))
    d = exp(r * h - sigma * sqrt(h))
    p_star = (exp(r * h) - d) / (u - d)

    allPaths = SharedArray{Float64}((n + 1, numPaths))

    @sync @distributed for j in 1:numPaths
        allPaths[1, j] = S
        for i in 2:(n + 1)
            if rand() < p_star
                allPaths[i, j] = allPaths[i - 1, j] * u
            else
                allPaths[i, j] = allPaths[i - 1, j] * d
            end
        end
    end

    return allPaths
end

numPaths = 50
parallelPaths = createPathParallel(S, r, sigma, T, n, numPaths)

plot(xlabel = "Time (years)", ylabel = "Stock Price",
     title = "50 параллельных траекторий")
for j in 1:numPaths
    plot!(time_axis, parallelPaths[:, j], legend = false, alpha = 0.4)
end
plot!()

```



Рис. 3.35: Пункт с) задачи № 3

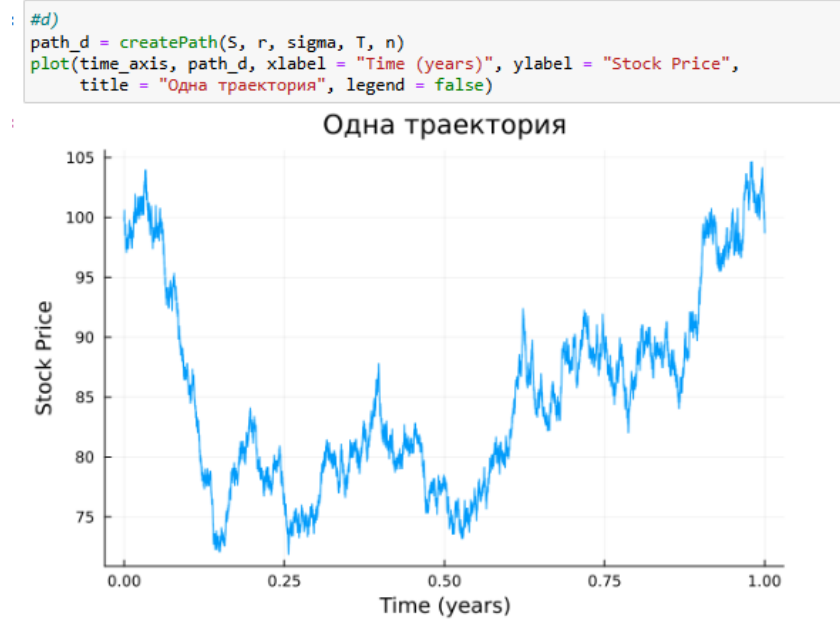


Рис. 3.36: Пункт d) задачи № 3

4 Выводы

В результате выполнения данной лабораторной работы я освоила специализированные пакеты Julia для обработки данных.